阿里AI技术栈与电商规则调研笔记

阿里AI技术栈与电商规则调研笔记

阿里云AI开放平台概览

阿里云的AI技术开放平台提供了一系列成熟的人工智能能力，以API服务形式对外开放。其中，“阿里云视觉智能开放平台”聚焦计算机视觉领域，涵盖**13大类目、上百项视觉AI能力**，包括人脸与人体识别、文字识别OCR、商品图像理解、内容安全审核、图像识别、图像生成与编辑、图像分割抠图、视觉搜索、图像分析处理、目标检测、视频理解、视频生成、视频分割等。这些能力广泛适用于智慧城市、安防监控、数字营销、身份验证、娱乐互动、移动应用等场景，开发者和企业可按需选用相关API封装成解决方案或产品。所有AI能力均通过**标准化的API接口**提供，开发者只需按统一规范调用即可将视觉智能能力集成到自身应用中。

值得注意的是，视觉智能开放平台基于阿里巴巴内部多年视觉AI技术实践，旨在以**高易用性和普惠性**为特色，让开发者以低门槛使用先进的视觉AI能力，加速业务智能化。平台会持续迭代新增更多AI能力，开发者可关注官方动态获取最新能力发布信息。

应用场景：阿里云视觉智能开放平台面向智慧城市、安防监控、数字营销、金融身份认证、互联网娱乐、移动应用等行业，提供普惠易用的视觉AI能力服务。平台涵盖人脸人体、文字识别、内容审核、目标检测、视觉搜索等 13 个类目、上百种视觉AI能力，开发者可根据业务需要选择相应能力，通过标准化的 API 接口快速接入并构建应用。

接入流程：首次使用视觉智能开放平台，可按照以下步骤完成API接入配置：

1. **注册账号及实名认证**：注册阿里云账号并完成实名认证（个人或企业认证）。账号需实名后才能购买和使用视觉智能开放平台服务，其中人脸人体类目仅支持已实名认证（个人或企业）的用户开通使用。企业用户建议进行企业认证以获取完整功能。
2. **开通服务能力**：登录视觉智能开放平台控制台的“能力广场”，选择所需的AI能力类目并单击“立即开通”进入购买页面。在购买页选择地域和对应能力，勾选服务协议后确认开通。**说明**：开通某一类目下任意AI能力，即表示该类目下所有能力均已开通且可调用（开通本身不收费）。公测期能力可免费使用，付费能力的计费方式请参考后文计费说明或官方计费文档。
3. **创建AccessKey**：在阿里云控制台获取访问密钥（AccessKey ID和AccessKey Secret），用于API鉴权调用。使用子账号时需确保已为其授予 **AliyunVIAPIFullAccess** 权限，否则无法调用视觉智能开放平台的API。访问密钥的创建方法详见官方指南。
4. **准备请求数据**：根据API要求准备待分析的图像/视频数据。**注意**：强烈建议将文件存储于阿里云OSS（华东2-上海）并使用其URL进行调用，以获得最佳兼容性和性能。如果文件在本地或非上海地域OSS，请参考“文件URL处理”文档，将其转换为上海地域的临时URL或Base64格式。文件URL中不能包含中文或空格等特殊字符，若有则需先进行URL编

码。同时请确保上传的图片符合接口支持的格式和大小限制（详见各能力的输入限制说明）。

5. **接口调试（可选）**：使用 OpenAPI Explorer 调试工具在线调试接口，免除手动计算签名的步骤 。通过Explorer可查看完整的请求参数、返回结果，并自动生成对应语言的SDK调用示例代码。
6. **开发集成**：在正式开发中，**优先使用阿里云提供的SDK** 调用视觉智能开放平台API。选择适合的语言SDK并安装对应的SDK包，然后参考官方示例代码进行调用开发。SDK支持Java、Python、Go、Node.js、PHP、C#六种语言，可在下文“SDK下载与配置”部分获取各语言的安装方法。若目标语言暂无SDK覆盖，则可采用API签名机制直接发起HTTP请求，详见下文签名说明。

以下将以几个典型视觉AI能力为例，说明具体API的请求构造、参数说明和返回结果格式，以及错误码含义等，帮助您快速完成接入。

身份证识别 API 调用示例

功能描述：文字识别（OCR）类目的身份证识别接口（`RecognizeIdentityCard`）可以识别中华人民共和国第二代居民身份证图片中的关键字段信息，包括姓名、性别、民族、身份证号、出生日期、住址、有效期限起止日期、签发机关，并输出身份证人像面与国徽面的区域位置及人脸置信度等信息。目前仅支持识别中国大陆身份证，不支持港澳台地区身份证。

应用场景：身份认证场景，例如用户在线注册时上传身份证照片，系统通过OCR接口提取证件信息，自动完成表单填写或实名验证。

输入限制：支持的图片格式为 JPEG、JPG、PNG、BMP、GIF，文件大小不超过3MB，分辨率在15×15至4096×4096像素之间。URL中不得包含中文字符。若图片不在指定格式或大小范围内，接口可能返回相应的错误码（详见后述错误码）。

请求接口：通过 HTTP/HTTPS 调用 `ocr.cn-shanghai.aliyuncs.com` 域名下的 `RecognizeIdentityCard` 接口，需使用GET方法或POST表单提交公共请求参数和业务参数。请求时除公共参数外，需要提供以下业务参数：

- **Action**：接口名，固定为 `RecognizeIdentityCard` 。
- **ImageURL**：待识别身份证照片的URL。推荐使用上海地域OSS的外链URL；如果文件在本地或非上海OSS，可参考前述文件URL处理方式获取可访问的URL。
- **Side**：身份证正反面标识，字符串，可选参数。取值为 `face` 或 `back` ，分别表示人像面和国徽面。不传该参数时系统可能自动判断或只识别人像面。

示例请求URL构造如下：

复制代码

```
http(s)://ocr.cn-shanghai.aliyuncs.com/?Action=RecognizeIdentityCard
&ImageURL=http://viapi-test.oss-cn-shanghai.aliyuncs.com/viapi-
3.0domepic/ocr/RecognizeIdentityCard/sfz1.jpg
&Side=face
```

```
&<公共请求参数>
```

返回数据：接口调用成功时，返回HTTP 200状态码，结果数据包括请求ID和识别出的身份证信息。返回参数说明如下：

- **RequestId** (String)：请求标识符，用于唯一标记一次请求。
- **Data** (Object)：识别结果数据容器。
  - **FrontResult** (Object)：当请求为人像面（`Side=face`）时返回的人像面识别结果，包括如下字段：
    - **Name** (String)：姓名
    - **Gender** (String)：性别
    - **Nationality** (String)：民族
    - **BirthDate** (String)：出生日期，YYYYMMDD 格式
    - **Address** (String)：住址
    - **IDNumber** (String)：公民身份证号
    - **FaceRectangle** (Object)：身份证人像照片中人脸的位置矩形及尺寸（包含子字段 Center、Size、Angle 等，表示人脸框中心坐标、大小、高度宽度和旋转角度）。
    - **FaceRectVertices** (Array)：人像照片人脸区域的四个顶点坐标。顶点按左下、右下、右上、左上顺序给出。
    - **CardAreas** (Array)：身份证整张卡片区域的四个顶点坐标（左上、左下、右下、右上顺序）。
    - *... (其他可能字段如人脸特征置信度等，根据实际返回为准)*
  - **BackResult** (Object)：当请求为国徽面（`Side=back`）时返回的国徽面识别结果：
    - **Issue** (String)：签发机关
    - **StartDate** (String)：证件有效期起始日期YYYYMMDD
    - **EndDate** (String)：证件有效期截止日期YYYYMMDD

返回示例：正常返回的XML示例如下：

复制代码

```xml
<RecognizeIdentityCardResponse>
    <RequestId>A6175A03-624C-1976-908D-C8BE4C547C45</RequestId>
    <Data>
        <FrontResult>
            <Name>小明</Name>
            <Gender>男</Gender>
            <Nationality>汉</Nationality>
            <BirthDate>19880722</BirthDate>
            <Address>江苏省南京市浦口区天天小区1栋11号</Address>
            <IDNumber>320001198807220000</IDNumber>
```

```
            <FaceRectangle> ... </FaceRectangle>
            <FaceRectVertices> ... </FaceRectVertices>
            <CardAreas> ... </CardAreas>
        </FrontResult>
    </Data>
</RecognizeIdentityCardResponse>
```

对应的JSON格式示例：

复制代码

```
{
  "RequestId": "A6175A03-624C-1976-908D-C8BE4C547C45",
  "Data": {
    "FrontResult": {
      "Name": "小明",
      "Gender": "男",
      "Nationality": "汉",
      "BirthDate": "19880722",
      "Address": "江苏省南京市浦口区天天小区1栋11号",
      "IDNumber": "320001198807220000",
      "FaceRectangle": { ... },
      "FaceRectVertices": [ ... ],
      "CardAreas": [ ... ]
    }
  }
}
```

错误码：关于身份证识别接口的错误码信息，请参考文末“常见错误码”部分。

人脸检测与五官定位 API 调用示例

功能描述：人脸人体（facebody）类目的“人脸检测与五官定位”接口（`DetectFace`）可以检测输入图片中的所有人脸，并返回每张人脸的位置坐标、人脸数量、人脸姿态（三维欧拉角）、人脸关键点（共105个关键点）以及人脸质量评价等信息。该能力能够适应多种复杂场景，包括遮挡、光照变化、模糊、多姿态和噪声环境，即使单张图片中包含多个人脸也能精确定位每个人脸的五官特征点。

应用场景：人脸关键点检测是众多人脸应用的基础能力，可用于互动娱乐（如动态贴纸、人脸特效相机）、美颜拍照、表情识别、疲劳检测、三维人脸重建等场景。例如：

- *互动娱乐应用*：精确定位人脸五官轮廓，实现贴纸特效、小视频滤镜等功能。
- *人脸美颜*：利用高精度关键点进行人像美颜和瘦脸，应用于照片编辑、视频直播等。

- *智能分析*：通过面部关键点和姿态分析，用于智能医美规划、身份验证中的活体检测等高价值场景。

输入限制：支持JPEG、JPG、PNG、BMP格式图片，文件大小不超过30MB。图片分辨率需在32×32到8192×8192之间，且人脸区域不低于64×64像素。URL中不能包含中文字符或空格。若图片尺寸超过最大限制，请先行缩放处理至合规大小。

请求接口：通过HTTP(S)调用 `facebody.cn-shanghai.aliyuncs.com` 域名下的 `DetectFace` 接口。需要提供以下业务参数：

- **Action**：接口名，取值 `DetectFace` 。
- **ImageURL**：待检测的人脸图片URL。建议使用上海地域OSS的URL；如使用非上海OSS或本地图片，请参考文件URL处理方式。
- **Landmark**：是否返回人脸关键点坐标，Boolean类型，默认false。设为 true 则返回每张人脸的105个特征点坐标。
- **Quality**：是否返回人脸图像质量评分信息，Boolean类型，默认false。设为 true 则返回模糊度、光照、遮挡等质量评估结果。
- **Pose**：是否返回人脸姿态角度信息，Boolean类型，默认false。设为 true 则返回人脸的yaw/pitch/roll 三个角度值。
- **MaxFaceNumber**：最大人脸检测数，取值1~30，默认值1。如果图片中有人脸数量多于此值，则只返回置信度最高的前N个人脸信息（按FaceProbability从高到低排序）。

示例请求URL：

复制代码

```
http(s)://facebody.cn-shanghai.aliyuncs.com/?Action=DetectFace
&ImageURL=http://viapi-test.oss-cn-shanghai.aliyuncs.com/viapi-
3.0domepic/facebody/DetectFace/DetectFace1.png
&Landmark=true&Quality=true&Pose=true&MaxFaceNumber=1
&<公共请求参数>
```

返回数据：调用成功返回200状态，结果中包含RequestId和检测到的人脸信息列表。主要返回字段说明：

- **RequestId**：请求ID。
- **Data**：结果数据对象，包含此次检测的输出信息：
  - **FaceCount** (Integer)：检测出的人脸数量。
  - **FaceRectangles** (Array<Integer>)：人脸矩形框位置，格式为 `[left, top, width, height]` ，若检测出多张人脸则按顺序拼接多个矩形框参数。（坐标系以图像左上角为原点，left/top为人脸框左上角坐标，width/height为宽高）。
  - **FaceProbabilityList** (Array<Float>)：图像中各人脸的置信度列表，取值范围0~1。如果有多张人脸则返回如 `[p1, p2, ...]` ，值越接近1表示越可能为人脸且图像质量越高。

- **Landmarks** (Array<Float>)：人脸关键点坐标列表，每个人脸返回105对坐标（共210个数值），顺序依次为每个人脸的105个点 `(x0, y0, x1, y1, ... x104, y104)` 。
- **LandmarkCount** (Integer)：人脸特征点数量，固定为105。
- **LandmarkScore** (Array<Float>)：人脸关键点检测综合评分，(0,100]，推荐阈值85。每张人脸对应一个评分，分数越高表示关键点定位质量越好。
- **Pupils** (Array<Double>)：双瞳孔中心点坐标及半径，每张人脸返回6个值，格式为 `[左眼中心x, 左眼中心y, 左眼半径, 右眼中心x, 右眼中心y, 右眼半径]` 。
- **PoseList** (Array<Float>)：人脸三维姿态角，[yaw, pitch, roll] 顺序，每张人脸返回一组三个值。yaw为左右转角、pitch为上下仰角、roll为平面旋转角度，单位度，范围见文档说明。
- **Qualities** (Object)：人脸图像质量评价结果，包含以下各项子列表，每项按人脸返回一个值：
  - **ScoreList**：综合质量分，越高表示图片越利于识别。
  - **BlurList**：清晰度分数，越高表示人脸越清晰（模糊程度越低）。
  - **IlluList**：光照充足度分数，越高表示光照条件越好。
  - **MaskList**：遮挡（口罩）影响分数，越高表示人脸下半部分无遮挡。
  - **GlassList**：眼镜遮挡影响分数，越高表示人脸上半部分无遮挡（未戴眼镜或不影响识别）。
  - **NoiseList**：噪声影响分数，越高表示图像噪声越小。
  - **FnfList**：是否为人脸判定分数，越高表示确认为人脸的置信度越高。
  - **...** *(更多质量评估子项详见文档)*

返回示例：正常返回的XML示例：

复制代码

```
<DetectFaceResponse>
  <RequestId>26B5334B-FD8A-5994-A1DA-3CA8F7B25676</RequestId>
  <Data>
    <FaceCount>1</FaceCount>
    <FaceRectangles>358</FaceRectangles>
<FaceRectangles>141</FaceRectangles><FaceRectangles>207</FaceRectangles>
<FaceRectangles>255</FaceRectangles>
    <FaceProbabilityList>0.96</FaceProbabilityList>
    <PoseList>5.02</PoseList><PoseList>-3.95</PoseList>
<PoseList>2.41</PoseList>
    <Landmarks>381.1</Landmarks><Landmarks>201.72</Landmarks>...
<Landmarks>197.65</Landmarks>
    <LandmarkCount>105</LandmarkCount>
    <LandmarkScore>95.89</LandmarkScore>
    <Qualities>
        <ScoreList>99.92</ScoreList>
        <BlurList>99.21</BlurList>
```

```
        <FnfList>100</FnfList>
        <GlassList>100</GlassList>
        <IlluList>100</IlluList>
        <MaskList>99.53</MaskList>
        <NoiseList>99.74</NoiseList>
        <PoseList>100</PoseList>
    </Qualities>
  </Data>
</DetectFaceResponse>
```

对应的JSON格式示例：

复制代码

```
{
  "RequestId": "26B5334B-FD8A-5994-A1DA-3CA8F7B25676",
  "Data": {
    "FaceCount": 1,
    "FaceRectangles": [358, 141, 207, 255],
    "FaceProbabilityList": [0.96],
    "PoseList": [5.02, -3.95, 2.41],
    "Landmarks": [381.1, 201.72, 448.09, ... 197.65],
    "LandmarkCount": 105,
    "LandmarkScore": 95.89,
    "Qualities": {
      "ScoreList": [99.92],
      "BlurList": [99.21],
      "FnfList": [100],
      "GlassList": [100],
      "IlluList": [100],
      "MaskList": [99.53],
      "NoiseList": [99.74],
      "PoseList": [100]
    }
  }
}
```

错误码：关于人脸检测与五官定位接口的错误码说明，请参见文末“常见错误码”部分。

以图搜图（视觉搜索）API 调用示例

功能描述：视觉搜索类目的“搜索图片”接口（`SearchImage`）用于基于输入的一张图片，在预先构建的图库数据库中搜索相似的图片。常用于电商以图找货、图片去重等场景。*（注意：视觉搜索服务自2023年2月15日起不再开放新用户使用，新用户请改用新版“图像搜索”服务。已开通的旧用户仍可调用，如有问题可通过钉钉群咨询。）*

输入限制：支持查询图片格式为JPG/JPEG/PNG，大小不超过3MB，分辨率在128×128到1024×1024之间。图片URL中不能含中文字符。超出规格的输入可能导致错误返回（如InvalidImage.Resolution等错误码）。

计费说明：搜索图片接口为付费接口，按调用次数计费，具体价格和默认QPS限制请参考官方计费文档。

请求接口：通过HTTP(S)调用 `imgsearch.cn-shanghai.aliyuncs.com` 域名下的 `SearchImage` 接口。主要业务参数包括：

- **Action**：接口名，固定为 `SearchImage`。
- **DbName**：搜索的图片数据库名称。需提前在控制台或通过API创建并导入图片。示例值“default”。
- **ImageUrl**：待搜索的图片URL。建议使用上海地域OSS的URL，如果使用本地文件或非上海OSS链接，请参见文件URL处理文档来获取可访问的URL。
- **Limit**：返回相似结果的数量上限，取值1~200。例如填5表示返回最相似的5张图片信息。

示例请求URL：

复制代码

```
http(s)://imgsearch.cn-shanghai.aliyuncs.com/?Action=SearchImage
&DbName=default
&ImageUrl=http://viapi-test.oss-cn-shanghai.aliyuncs.com/viapi-
3.0domepic/imgsearch/SearchImage/SearchImage-ku8.png
&Limit=5
&<公共请求参数>
```

返回数据：调用成功返回RequestId和Data，其中Data包含相似图片列表等结果：

- **RequestId**：请求ID，用于跟踪请求。
- **Data**：结果数据对象：
  - **MatchList** (Array)：相似图片列表，每个元素包含：
    - **EntityId** (String)：图片实体ID（即加入图库时分配的唯一标识）。
    - **DataId** (String)：数据ID（用户自定义的图片编号，如上传时提供）。
    - **ImageUrl** (String)：相似图片的URL。注意：返回的URL即入库时提供的字符串，平台并不存储原始图片文件。如果入库时使用的是临时OSS URL，该URL可能已过期，建议使用EntityId在自有存储中索引图片。另返回URL为临时访问地址，有效期30分钟。

- **Score** (Float)：相似度得分，0~10，越大表示图片越相似。
- **ExtraData** (String)：用户自定义的附加信息（如果上传入库时提供了该字段）。

返回示例：正常返回的XML示例：

复制代码

```
<SearchImageResponse>
  <RequestId>A98DBCEB-45E2-5F5F-B5ED-F2340182FD87</RequestId>
  <Data>
    <MatchList>
      <EntityId>123456</EntityId>
      <DataId>1</DataId>
      <ImageUrl>http://vision-console.oss-cn-
shanghai.aliyuncs.com/.../image1.jpg?...Signature=8rj0%2Bxg%2Bni5NxQAcEQ
qISX5j1w****</ImageUrl>
      <Score>10</Score>
      <ExtraData>zhaoliu</ExtraData>
    </MatchList>
    <MatchList> ... 第二个相似结果 ... </MatchList>
    <MatchList> ... 第三个相似结果 ... </MatchList>
    <!-- 可能有多个MatchList -->
  </Data>
</SearchImageResponse>
```

对应JSON示例：

复制代码

```
{
  "RequestId": "A98DBCEB-45E2-5F5F-B5ED-F2340182FD87",
  "Data": {
    "MatchList": [
      {
        "EntityId": "123456",
        "DataId": "1",
        "ImageUrl": "http://vision-console.oss-cn-
shanghai.aliyuncs.com/.../image1.jpg?...Signature=8rj0%2Bxg%2Bni5NxQAcEQ
qISX5j1w****",
        "Score": 10,
        "ExtraData": "zhaoliu"
      },
      {
        "EntityId": "123457",
        "DataId": "2",
```

```
        "ImageUrl": "http://vision-console.oss-cn-shanghai.aliyuncs.com/.../image2.jpg?...Signature=%2Byceg79YwiShi1RWDSQ%2FSZcIec****",
        "Score": 1.1763,
        "ExtraData": "wangwu"
      },
      ...
    ]
  }
}
```

错误码：关于以图搜图接口的错误码，请参见文末“常见错误码”部分。

常见错误码说明与排查建议

视觉智能开放平台各接口可能返回的错误信息及原因如下。当API请求发生错误时，HTTP状态码一般为4xx或5xx，响应中会包含 `Code` 和 `Message` 字段指示错误类型。常见错误码列表如下：

- **403 AuthFailed**：授权失败，请检查RAM权限配置是否正确。调用视觉智能API需要授予账号 **AliyunVIAPIFullAccess** 权限。
- **400 ClientError.IllegalArgument**：请求参数不合法，请检查参数取值是否合法、所指代的资源是否存在等。
- **403 EntityNotExist.Role**：RAM权限不足。请为RAM用户添加 **AliyunVIAPIFullAccess** 权限后重试。
- **400 IllegalUrlParameter**：URL参数不合法。请检查提供的URL是否可正常访问；对于非上海OSS的图片链接，请参考文件URL处理文档进行处理。
- **500 InternalError / 586 InternalError.Algo**：服务器内部错误或算法处理错误。请稍后重试或联系技术支持。
- **500 InternalError.Timeout**：服务器处理超时。请稍后重试，如问题持续可通过钉钉群联系支持。
- **400 InvalidAccessKeyId.Inactive**：提供的AccessKey ID非法或已禁用，请确认AK是否有效。
- **404 InvalidAccessKeyId.NotFound**：AccessKey ID不存在，请核实调用时使用的AccessKey是否正确。
- **400 InvalidAccessKeySecret**：AccessKeySecret错误，请检查AK/SK填写是否正确。
- **400 InvalidAction.NotFound**：调用的Action不存在。请检查接口名称是否拼写正确，且调用的接口是否属于当前类目，访问域名是否匹配。

- **400 InvalidApi.ForbiddenInvoke**：接口调用受限。通常原因是调用了受限能力。对于受限能力，需要先在控制台申请开通或通过审批后方可调用。若非受限能力，另一可能是账号欠费，请排查账户余额。
- **403 InvalidApi.NotPurchase**：接口未购买/未开通。请先开通相应类目的服务后再调用。
- **400 InvalidApi.OutOfService**：接口已下线或不可用，可能是产品未开通或已停止服务。
- **400 InvalidFile.Content**：文件内容不合法或不符合要求。请确认文件格式、内容符合接口要求（例如图片完整可读，非空文件等）。
- **400 InvalidFile.Decode**：文件解码失败。请检查文件是否损坏或能否正常打开。
- **400 InvalidFile.Download**：文件下载失败。请检查本地网络以及文件URL的可访问性。对于非上海OSS链接，请参考文件URL处理文档。
- **400 InvalidFile.REGION**：文件链接地域错误。请确保使用上海地域的OSS链接，或参考文件URL处理将文件存储迁移到指定地域。
- **400 InvalidFile.Resolution**：文件分辨率超出限制。请检查图片分辨率是否在要求范围内，调整后重试。
- **400 InvalidFile.Type**：文件类型不受支持。请确认文件类型与接口要求一致（系统会根据文件内容判断类型而非扩展名）。
- **400 InvalidFile.URL**：文件URL非法或无法访问。请检查URL有效性，上海OSS链接请使用标准域名（不支持CDN加速域名）。
- **400 InvalidImage.Category**：输入图像与接口服务不符。例如调用行驶证识别接口却提供了非行驶证的图片。
- **400 InvalidImage.Content**：图像内容不合法或为空。可能是Base64编码无效、字符串为空等，请检查并确保提交正确的图像数据。
- **400 InvalidImage.Decode**：图片解码失败，请确认图片文件未损坏，能正常打开。
- **400 InvalidImage.Download**：图片下载失败，可能是网络问题或OSS链接过期。请检查链接有效性；URL中不支持空格和中文，必要时请进行URL编码。
- **400 InvalidImage.Gif**：GIF图片处理失败（可能文件损坏或格式不符合规范）。
- **404 InvalidImage.NotFoundFace**：图片中未检测到人脸。请确认上传的图片包含有人脸，且人脸尺寸不要过小。
- **400 InvalidImage.Region**：图片链接地域不符要求。请使用上海地域的OSS链接或按文档转换。
- **400 InvalidImage.Resolution**：图片分辨率超限，超出接口支持范围。请调整图像分辨率后重试。
- **400 InvalidImage.Timeout**：图片下载超时。请检查网络连接，或OSS临时URL是否未过期。
- **400 InvalidImage.Type**：图片类型错误，请提供接口支持的图片格式。
- **400 InvalidImage.Unsafe**：图片URL不安全（可能使用了非HTTP协议或不受信任的源）。

- **415 InvalidImage.UnsupportedMediaType**：提供的Image数据无法解析，可能文件格式不被支持。
- **400 InvalidImage.URL**：图片链接非法或无效。请检查链接可访问性；上海OSS链接需使用原始域名（不支持自定义域名）。
- **400 InvalidImageType**：无效的图片类型。请检查输入是否为有效图像格式。
- **400 InvalidOutputFormat**：无效的输出格式参数。
- **400 InvalidParameter**：参数不合法或缺失。请根据报错信息检查请求参数格式和取值。
- **400 InvalidParameter.BadRequest**：参数格式错误，通常是参数值不正确或存在多余空格等。请严格按API文档要求提供参数。
- **400 InvalidParameter.NotFound**：参数取值有误，可能指定资源不存在。请根据报错信息核对参数内容。
- **400 InvalidParameter.TooLarge**：参数过大，可能文件大小超限或文字内容超长。请确保提交内容在API限制范围内。
- **403 InvalidRamRole**：RAM角色无权限，请为其授予AliyunVIAPIFullAccess权限后再试。
- **400 InvalidResult**：返回结果异常。对于OCR场景表示识别失败或异常，请检查图片质量，如过于模糊或内容缺失等。
- **400 InvalidSide**：`Side` 参数不合法。请按文档要求填写正确的 `Side` （如身份证识别接口仅接受 "face" 或 "back"）。
- **400 InvalidTimeStamp.Expired**：时间戳过期。请确保请求中的时间戳在服务器时间15分钟以内。
- **400 MissingAccessKeyId**：请求缺少AccessKey ID参数。请检查是否正确传递了 `AccessKeyId` 。
- **400 MissingAssureDirection**：缺少必需的 `AssureDirection` 参数（某些接口特有参数），请检查参数完整性。
- **400 MissingImageURL**：缺少图片URL参数，请确保提交了 `ImageURL` 。
- **400 MissingMinHeight**：缺少最小高度参数，请检查请求是否遗漏该参数。
- **400 MissingOutputFormat**：缺少输出格式参数。
- **400 MissingSide**：缺少身份证正反面 `Side` 参数。
- **400 MissingSkipDetection**：缺少SkipDetection参数。
- **400 MissingUseFinanceModel**：缺少UseFinanceModel参数。
- **400 MissingParameter**：参数缺失，请检查是否遗漏必填参数。
- **400 ParameterError**：参数错误，请检查参数取值是否正确。
- **503 ServiceUnavailable**：服务不可用，可能是临时服务器问题。如果返回了RequestId，请提供给技术支持排查。
- **400 SignatureDoesNotMatch**：签名不匹配，签名计算错误。请确认使用正确的AccessKey，并严格按照请求签名算法计算签名。
- **400 SignatureNonceUsed**：随机数 `SignatureNonce` 重复使用。请确保每次请求使用唯一的随机数值。

- **400 Throttling / 400 Throttling.User**：请求被限流。表示短时间内调用频率超过限制。对于付费接口请考虑购买更高QPS配额，免费接口如需更高限额可联系阿里云技术支持。
- **408 Timeout**：请求超时。请检查网络连接，并可重试请求。
- **400 Unauthorized**：未经过授权调用。通常是RAM用户未授予权限，请为其添加AliyunVIAPIFullAccess策略。

处理建议：开发者在收到错误码时，可根据以上描述定位问题原因并进行处理。常见错误类型及建议如下：

- **URL类错误**：多由文件URL不符合要求导致，例如URL非法或不可达。建议按文档确保使用OSS公网URL，并进行必要的编码处理。
- **图像加载错误**：多因图像URL无效或地域不符导致，如下载失败、区域错误。平台推荐使用华东2（上海）区域的OSS来存储文件。
- **参数类错误**：通常由于缺少必填参数或参数格式错误。请对照API文档的请求参数列表检查输入是否完整且正确。
- **系统类错误**：如遇内部错误或超时，建议稍候重试；若持续出现，可携带RequestId通过钉钉群联系官方支持。

调用限制与QPS说明

QPS（每秒查询率）限制：视觉智能开放平台针对各接口均设有默认的QPS限制，即每秒允许的最大调用次数。默认QPS值可在官方计费详情页查询，不同能力可能不同。例如，某接口默认QPS=2，则在1秒内最多允许2次成功调用，第3次及以上调用将在当秒内被拒绝并返回限流错误，错误码一般为 **Throttling** 或 **Throttling.User**，提示“调用被限流，请稍后再调用，或者购买QPS提升调用限制”。被限流的请求不会计费。

如果您的业务调用频率可能超出默认QPS限制，有以下两种扩容方式：

- **购买预付费资源包**：资源包附带一定QPS额度，可提升当日QPS上限。各能力对应的资源包及提升的QPS值详见计费详情的资源包章节。购买资源包后通常在次日早8点生效新的QPS限制。
- **购买预付费QPS**：直接购买额外的QPS配额，在购买时段内无限次调用且免除按次计费。预付费QPS购买后即时生效。

部分能力支持**QPS升配**（超出默认上限的更高QPS），如人脸人体类目最高可扩展至50 QPS，目标检测/视频生产类最高100 QPS等。如业务需要更高并发，可联系阿里云技术支持协助升配。

在按量计费、资源包、预付费QPS并存时，系统扣费顺序为：先消耗预付费QPS（不限次），其余调用再按资源包点数扣减，资源包耗尽后最终按量计费。当预付费QPS到期后，QPS限制会自动降为资源包对应的默认值；资源包用尽或过期则降为按量计费的默认QPS。

开发者可在视觉智能开放平台控制台的能力详情页面查看当前能力的QPS配额和使用情况 。若频繁出现 **Throttling** 错误且业务需要更高QPS，请考虑购买相应的扩容服务，以确保高并发场景下接口调用的稳定性。

SDK下载与配置（多语言支持）

阿里云视觉智能开放平台提供多语言SDK，方便开发者以所熟悉的语言集成API调用。目前官方SDK支持 **Java**、**Python**、**Go**、**Node.js**、**PHP**、**C#** 六种主流语言。开发者可根据项目需要选择对应语言的SDK并安装使用。各语言SDK获取和配置方法如下：

- **Java SDK**：支持Java 8及以上环境。通过Maven引入依赖或直接下载Jar包使用。不同AI类目对应不同的SDK模块，例如：
  - 人脸人体类（facebody）：Maven坐标为 `com.aliyun:facebody20191230`，可访问Maven中央库获取最新版本。GitHub源码链接：Aliyun Java SDK Facebody。
  - 文字识别类（ocr）：Maven坐标 `com.aliyun:ocr20191230`。GitHub源码：Aliyun Java SDK OCR。
  - 视觉搜索类（imgsearch）：Maven坐标 `com.aliyun:imgsearch20200320`。
  - ...（完整列表参见官方文档SDK参考的Java章节，各类目的SDK名称及下载链接）。

引入依赖后，在代码中配置客户端：设置AccessKey、Secret和对应接口的Endpoint即可使用。示例代码见下文。

- **Python SDK**：支持Python 3及以上版本。推荐使用pip安装所需的SDK库。常用能力对应的pip包名称如下：
  - 人脸人体：`alibabacloud_facebody20191230`
  - 文字识别：`alibabacloud_ocr20191230`
  - 商品理解：`alibabacloud_goodstech20191230`
  - 内容审核：`alibabacloud_imageaudit20191230`
  - 图像识别：`alibabacloud_imagerecog20190930`
  - 图像生产：`alibabacloud_imageenhan20190930`
  - 分割抠图：`alibabacloud_imageseg20191230`
  - 目标检测：`alibabacloud_objectdet20191230`
  - 图像分析处理：`alibabacloud_imageprocess20200320`
  - 视觉搜索：`alibabacloud_imgsearch20200320`
  - 视频理解：`alibabacloud_videorecog20200320`
  - 视频生产：`alibabacloud_videoenhan20200320`
  - 视频分割：`alibabacloud_videoseg20200320`
  - 异步任务管理：`alibabacloud_viapi20230117`

（以上列出了常用能力的SDK包名。安装时按需选择对应的包，执行 `pip install <包名>` 即可）。安装完成后，直接 `import` 相应模块即可在代码中使用SDK。也可通过源码仓库获取：例如GitHub上的 facebody-20191230 对应 pip 包 `alibabacloud_facebody20191230`。

- **Go SDK**、**Node.js SDK**、**PHP SDK**、**C# SDK**：平台同样提供这些语言的SDK包，可通过Go Modules、npm、Composer、NuGet等包管理器获取。包名称一般包含对应服务名称和版本号。例如Node.js的facebody SDK包名为 `@alicloud/facebody20191230` 。请参考阿里云官方文档中各语言的SDK使用指南获取详细安装和调用方法。
- **前端/移动端直接调用**：视觉智能开放平台也支持Web前端、小程序、移动端直接调用的方案。例如Web前端和小程序可通过前端SDK进行匿名鉴权调用，移动端(Android/iOS)可使用专用SDK。然而，由于直接在客户端包含AccessKey存在安全风险，**强烈建议将调用逻辑放在后端服务器**，通过后端转发请求以保护密钥安全。
- **签名直调模式**：对于未提供官方SDK的语言或特殊场景，可使用HTTP API签名方式直接调用。开发者需要自行按照阿里云API签名算法将请求参数签名并调用对应的Endpoint。签名计算请参见《请求签名》文档 。在签名直调时，同样需要注意文件参数的处理，如非上海OSS的文件需先转换链接。

在使用SDK前，请确保**不要将AccessKey ID/Secret硬编码在客户端应用中**，以免泄露。建议通过环境变量或服务器下发临时凭证的方式配置凭据。例如，在本地开发或服务器环境中，可将 `ALIBABA_CLOUD_ACCESS_KEY_ID` 和 `ALIBABA_CLOUD_ACCESS_KEY_SECRET` 配置为环境变量，SDK会自动读取。

示例代码（Python 与 Java）

下面提供使用SDK调用视觉智能开放平台API的示例代码片段，以Python和Java语言为例，演示如何发起请求并处理返回结果。以“人脸检测与五官定位”能力为例：

Python调用示例：使用官方Python SDK调用DetectFace接口。

复制代码

```python
# -*- coding: utf-8 -*-
# 安装SDK: pip install alibabacloud_facebody20191230
import os
from alibabacloud_facebody20191230.client import Client as
FacebodyClient
from alibabacloud_facebody20191230.models import DetectFaceRequest
from alibabacloud_tea_openapi.models import Config

# 配置Client
config = Config(
    access_key_id = os.environ.get('ALIBABA_CLOUD_ACCESS_KEY_ID'),
    access_key_secret =
os.environ.get('ALIBABA_CLOUD_ACCESS_KEY_SECRET'),
    endpoint = 'facebody.cn-shanghai.aliyuncs.com',
    region_id = 'cn-shanghai'
)
client = FacebodyClient(config)
```

```
# 构造请求
request = DetectFaceRequest(
    image_url = 'http://viapi-test.oss-cn-shanghai.aliyuncs.com/viapi-3.0domepic/facebody/DetectFace/DetectFace1.png',
    landmark = True,
    quality = True,
    pose = True,
    max_face_number = 1
)

# 发起请求并获取响应
try:
    response = client.detect_face_with_options(request, None)  # RuntimeOptions可为None
    print(response.body)                # 输出完整结果
except Exception as e:
    print(e)                            # 输出错误信息
    # 如需进一步处理错误，可输出 e.code 等属性
```

运行上述Python代码前，请确保已将有效的AccessKey配置为环境变量。调用成功时，`response.body` 将包含与前文返回示例类似的JSON结果，其中 `Data.FaceCount` 、 `Data.Landmarks` 等字段提供了检测到的人脸数量和关键点信息。若调用出错，将进入异常分支并输出错误码，开发者可据此排查问题。

Java调用示例：使用Java SDK调用DetectFace接口。首先通过Maven依赖导入 **facebody20191230** 模块，然后使用如下方式初始化客户端并发起请求：

复制代码

```
// 引入SDK依赖 (确保facebody20191230版本 >= 3.0.7)，初始化Client
import com.aliyun.facebody20191230.Client;
import com.aliyun.facebody20191230.models.*;
import com.aliyun.teaopenapi.models.Config;
...
Config config = new Config()
    .setAccessKeyId("<Your AccessKeyId>")
    .setAccessKeySecret("<Your AccessKeySecret>");
config.endpoint = "facebody.cn-shanghai.aliyuncs.com";
Client client = new Client(config);

// 构造请求
DetectFaceRequest request = new DetectFaceRequest()
    .setImageURL("http://viapi-test.oss-cn-shanghai.aliyuncs.com/viapi-3.0domepic/facebody/DetectFace/DetectFace1.png")
    .setLandmark(true)
```

```
    .setQuality(true)
    .setPose(true)
    .setMaxFaceNumber(1);

// 发起请求
try {
    DetectFaceResponse response = client.detectFaceWithOptions(request,
null);
    System.out.println(response.getBody().getData().getFaceCount());
// 示例：输出检测到的人脸数

System.out.println(response.getBody().getData().getLandmarkCount()); //
输出特征点数量
    // ... 可进一步处理 response 获取所需信息
} catch (Exception e) {
    e.printStackTrace();
    // 输出错误码
    if (e instanceof TeaException) {
        System.out.println(((TeaException)e).getCode());
    }
}
```

上述Java代码片段演示了创建Client、设置请求参数并调用的过程。调用成功后，可从 `response.getBody()` 中提取所需结果，例如人脸数、关键点坐标等；若失败则会抛出异常，开发者可捕获 `TeaException` 以获取错误码。

通过以上示例可以看出，使用SDK能大幅简化签名和请求构造流程，只需关注业务参数的设置和结果处理。在实际应用中，请根据需要调整参数和结果处理逻辑，结合前述**错误码**和**调用限制**说明完善异常处理和重试策略，确保应用的健壮性和稳定性。

阿里云平台API接入流程

阿里云 OpenAPI 是各云产品对外提供的统一接口，开发者可以通过调用 OpenAPI 实现对云资源的控制和管理。所有 OpenAPI 的调用遵循阿里云统一规范，下面从各个方面详细介绍阿里云技术开放平台的 API 接入方法和注意事项。

阿里云 OpenAPI 的统一调用规范：
阿里云 OpenAPI 具有两种主要的调用风格：RPC 风格和 RESTful 风格（ROA）。无论哪种风格，调用时都需要确定**服务接入点（Endpoint）**，使用 HTTPS 协议发送请求（为保证安全性，推荐 HTTPS）。绝大多数阿里云云产品采用 RPC 风格，即通过指定 Action 名称和请求参数调用对应接口；部分产品（如对象存储OSS等）采用 RESTful 风格，通过 REST API 路径进行资源操作。不同产品的 API 风格是统一且固定的，每个 API 仅支持特定的请求格式。一般调用一个 OpenAPI 需要关注：目标服务的 Endpoint 地址、是否需要身份认证、使用什么 HTTP 方法（GET/POST/PUT/DELETE 等）、请求参数、以及返回结果格式。若 API 需要身份认证，则请求中必须包含有效的凭证信息（如 AccessKey 签名）；如果是公开

匿名调用的 API，则不需要额外认证。阿里云所有 OpenAPI 无论成功或失败，都会返回统一结构的结果，其中包含本次请求的唯一标识 RequestId。开发者可以利用 RequestId 来定位问题或者与阿里云技术支持联系。此外，OpenAPI 调用支持返回 JSON 或 XML 格式的数据，用户可通过请求参数指定返回格式，默认为 JSON。请注意所有请求和返回均采用 UTF-8 编码。

请求结构说明（公共参数 + 业务参数）：
一次完整的 API 请求通常由以下部分组成：

- **协议和方法**：使用 HTTPS 协议，通过 GET 或 POST 方法发起请求。GET 方法时，请求参数放在 URL Query 中；POST 方法时，可以将参数放在请求体或 Query 中，具体参照接口要求。如果 API 同时支持 HTTP 和 HTTPS，推荐使用 HTTPS。
- **服务地址（Endpoint）**：即请求的目标域名和可选路径。每个云产品在各区域都有其 OpenAPI Endpoint，可在产品文档中查询。例如 ECS 在华东2（上海）的服务接入地址为 `ecs.cn-shanghai.aliyuncs.com` 。RESTful 风格的 API，Endpoint 后可能还需包含具体资源路径和路径参数。
- **公共请求参数**：每个 OpenAPI 调用都必须包含的一组参数，用于身份校验、协议控制等。常见公共参数包括：
  - `AccessKeyId` ：访问密钥 ID，用于标识请求发起者。
  - `Timestamp` ：请求时间戳，采用 ISO8601 UTC 时间格式，例如 `2023-03-07T12:00:00Z` 。通常要求时间戳在服务器当前时间 ±15分钟或 ±30分钟内，超出会被认为请求过期。
  - `SignatureNonce` ：唯一随机数，每次请求需不同，防止重放攻击。
  - `Signature` ：请求签名结果串，用于验证此次请求的身份。签名的计算方法见后文。
  - `SignatureMethod` ：签名算法名称，例如 `HMAC-SHA1` （V2版本签名）或 `ACS3-HMAC-SHA256` （V3版本）等。对于老版RPC签名，此参数固定为 HMAC-SHA1。
  - `SignatureVersion` ：签名算法版本，例如 `1.0` 。
  - `Format` ：返回数据格式，不传默认 XML，一般可选 `JSON` 或 `XML` 。现阶段多数产品默认 JSON。
  - `Version` ：API 的版本号，格式为日期 `YYYY-MM-DD` ，表示该接口所属的产品版本。必须提供正确的版本才能调用。
- **接口名称和业务参数**：对于 RPC 风格，需提供参数 `Action` 表示调用的具体接口名，以及该接口要求的其他业务参数（如实例ID、地域等）。例如调用 ECS 的查询实例列表接口，Action 可以是 `DescribeInstances` 且需提供 `RegionId` 等参数。对于 RESTful 风格，业务参数可能在 URL 路径、Query 或请求体中，例如通过 RESTful 路径 `GET /resource/{id}` 来操作特定资源。无论哪种风格，业务参数的具体定义需参考对应 API 文档或在阿里云 OpenAPI 开发者门户查询。

签名机制详解（签名算法、编码规范、构造规则）：
为了保证调用请求的合法性和防止篡改，所有需要身份认证的 OpenAPI 请求都必须按指定算法对请求进行数字签名。阿里云目前主要有两套签名机制：V2版本（不推荐）和改进的 V3 版本。下面分别介绍：

- **V2 版本签名机制（HMAC-SHA1，RPC/ROA 签名1.0）：** V2签名在请求中通过参数传递签名结果。计算签名时，需要首先收集并规范化请求参数，然后生成待签名字符串并计算 HMAC。具体步骤如下：

  a. **排序参数并编码**：将请求所有参数（包括公共参数和接口业务参数，但不含 `Signature` 本身）按参数名的字典序排序。然后按URL编码规范（RFC 3986）编码参数名称和值，拼接成 `"名称=值"` 形式。编码规则为：英文字母(A-Z, a-z)、数字(0-9)以及 `- _ . ~` 不编码，空格编码为 `%20`，星号 `*` 编码为 `%2A` 等。将所有编码后的键值对用 `&` 连接，即得到规范化的查询字符串 `CanonicalizedQueryString`。

  b. **构造待签名字符串**：按照固定格式将HTTP请求方法、编码后的资源路径和上一步生成的规范化查询字符串拼接成待签名字符串 `StringToSign`。格式为：`StringToSign = HTTPMethod + "&" + encodeURIComponent("/") + "&" + encodeURIComponent(CanonicalizedQueryString)`。其中 `HTTPMethod` 是请求使用的HTTP动词（如 `"GET"`），`"/"` 表示资源路径（针对RPC风格所有请求都是对根路径的操作），各部分再经URL编码后拼接。

  c. **计算签名值**：使用您的 AccessKeySecret，对上一步的 `StringToSign` 进行 HMAC-SHA1 运算，并将结果进行 Base64 编码，得到签名值。注意此处 HMAC 计算时密钥为 AccessKeySecret 后加上一个字符“&”的字节串（阿里云 V2 签名算法固定要求的约定），再对 StringToSign 进行摘要计算。伪代码表达式如下：

复制代码

```
Signature = Base64( HMAC_SHA1( AccessKeySecret + "&", UTF-8-Encoding-Of(StringToSign) ) )
```

4. **签名值加入请求**：将计算出的 Signature 再进行一次 URL 编码，然后以请求参数 `Signature` 的形式添加到请求URL或请求体中。这样服务器收到请求时即可根据请求内容计算签名并与 `Signature` 参数比对，验证身份。

按上述算法签名后，完整的 RPC 请求示例形式如下（为便于阅读添加换行）：

复制代码

```
https://ecs.cn-beijing.aliyuncs.com/?Action=DescribeDedicatedHosts
&Format=JSON
&Version=2014-05-26
&AccessKeyId=testid
&Signature=9NaGiOspFP5UPcwX8Iwt2YJXXuk%3D
&SignatureMethod=HMAC-SHA1
&SignatureNonce=edb2b34af0af9a6d14deaf7c1a5315eb
&SignatureVersion=1.0
&Timestamp=2023-03-13T08%3A34%3A30Z
&RegionId=cn-beijing
```

上述URL中各参数含义如前所述，Signature 为计算后的签名结果。服务端验证签名通过后，即会处理请求并返回结果。

提示： V2版本签名方式目前已不再推荐，官方建议使用新版的 V3 签名机制。在您已有的调用中，完全可以将签名算法无缝升级到 V3，以获得更高的安全性和一致性。

- **V3 版本签名机制（ACS3-HMAC-SHA256）：** V3签名将身份校验信息放入 HTTP 请求头部（Authorization 等），采用更安全的 HMAC-SHA256 算法，且覆盖了请求的关键要素（包括消息体内容摘要），可以防止消息被篡改。当不便使用SDK时，可通过自行计算V3签名直接以HTTP请求调用阿里云API。V3签名的概要流程如下：

> a. **构造规范化请求字符串（Canonical Request）**：提取HTTP请求的关键信息，按照定义的顺序和格式串联。具体包括：请求方法、请求路径、查询字符串（需将 Query 中参数名和值进行编码并按字典序连接为 `key=value&` 形式）、必要的请求头（见下文）、以及请求正文的 Hash 值等。将这些元素组合并计算 SHA256 摘要，得到 *hashedCanonicalRequest*。
> b. **构造待签名字符串（StringToSign）**：将签名算法标识和上一步得到的 hashedCanonicalRequest 按照指定格式拼接。格式一般为：

```
StringToSign = "<算法标识>\n<hashedCanonicalRequest>"
```

其中算法标识对于 V3 签名固定为 `"ACS3-HMAC-SHA256"`，字符串中使用换行符 `\n` 分隔。

> c. **计算签名值**：使用 AccessKeySecret 对 StringToSign 进行 HMAC-SHA256 运算，取结果的16进制表示（32字节哈希转化为64位十六进制字符串）作为签名值。伪代码：

```
Signature = HexEncode( HMAC-SHA256( AccessKeySecret, UTF-8-Encoding-Of(StringToSign) ) )
```

。注意与 V2 不同，这里不再在密钥后加符号 `&`，直接使用 AccessKeySecret 本身参与 HMAC。

4. **将签名添加到请求**：构造 HTTP “Authorization” 头，格式为：

`Authorization: ACS3-HMAC-SHA256 Credential=<AccessKeyId>,SignedHeaders=<SignedHeaders>,Signature=<Signature>`

其中，Credential 为您的 AccessKeyId，SignedHeaders 是参与签名的请求头列表，Signature 即步骤3计算的签名值。最后将该 Authorization 头添加到请求中。在请求头中还需包括 SignedHeaders 列出的各项（详见下文）。

V3签名请求所需的公共头信息： 使用 V3 签名方式时，请求必须携带一些额外的HTTP头以参与签名计算。主要包括：

- `x-acs-date`： 请求的时间戳，UTC 时间的 ISO8601 表示，要求为请求发起前15分钟内的时间。例如 `"2023-10-26T10:22:32Z"` 。
- `x-acs-signature-nonce`： 唯一随机字符串，每次请求都需不同，以防重放。
- `x-acs-version`： API版本号，如 `"2014-05-26"` 。
- `x-acs-action`： 要调用的API名称，如 `"RunInstances"` 。
- `host`： 服务域名，如 `"ecs.cn-shanghai.aliyuncs.com"` 。
- `x-acs-content-sha256`： 请求正文的 SHA256 摘要并经 HEX 编码的字符串。对于无请求体的GET请求，可取固定值 e3b0c442...（SHA256空串）。
- 此外，若请求有非空的请求体，一般还需提供 `Content-Type` （如 application/json）和 `Content-MD5` 等头用于标识内容类型和完整性。

将上述必要头域加入请求并正确计算 Authorization 签名后，即可完成 V3 请求的组装。服务端会验证 Authorization 中的 Signature 是否正确、各时间戳/Nonce 是否有效等，从而鉴权请求身份合法后再处理请求。V3 签名示例中，各步骤最终得到的 Authorization 头类似如下：

复制代码

```
Authorization: ACS3-HMAC-SHA256
Credential=YourAccessKeyId,SignedHeaders=host;x-acs-action;x-acs-content-sha256;x-acs-date;x-acs-signature-nonce;x-acs-version,Signature=0123456789abcdef...
```

相比 V2，V3 签名使用更强的哈希算法（SHA256）且保护了请求头和正文不被篡改，安全性更高。建议在新项目中使用 V3 签名调用，对于老项目也可平滑升级。需要注意，部分云产品由于使用自有网关，签名算法可能与通用规范略有不同，例如日志服务（SLS）和对象存储（OSS）有各自特殊的签名规则，调用这类产品时请参考它们的文档。整体而言，大部分阿里云云产品的 OpenAPI 均支持 V3 签名机制。

身份认证（AccessKey 与安全建议）：
调用需要鉴权的阿里云 API 时，必须提供有效的身份凭证。阿里云采用 AccessKey 来识别和验证用户身份。**AccessKey** 是由 AccessKey ID 和 AccessKey Secret 构成的一对密钥，用于 API 调用的身份验证。AccessKey ID 用于标识用户身份，AccessKey Secret 用于校验签名字符串的密钥。每次 API 请求都会使用 AccessKey Secret 对请求参数计算签名，服务端通过验证签名来确认请求者身份和权限。需要注意的是，AccessKey 仅用于编程访问（API、SDK、CLI 等），不能用于登录控制台。

AccessKey 获取与管理： 新用户可以在阿里云控制台上创建 AccessKey。出于安全考虑，**强烈不建议直接使用主账户（阿里云账号）的 AccessKey**。主账号 AK 拥有该账号下所有资源的完全权限，一旦泄露风险巨大。最佳实践是创建一个权限受限的 RAM 子用户，并为其分配所需的最小权限，然后为该 RAM 用户创建 AccessKey 用于 API 调用。创建 RAM 用户

AccessKey 可以通过 RAM 控制台，在RAM用户的“安全凭证”或“访问密钥”区域点击创建获取。**AccessKey Secret 只在创建时明文显示一次**，以后无法直接查询。请务必妥善保存 Secret，如果遗失只能重置创建新的。另外，每个 RAM 用户最多可拥有2对 AccessKey；如不再需要某个 AccessKey，应及时删除或禁用以降低泄露风险。

安全使用 AccessKey 的建议：

- **不要将 AccessKey 明文硬编码在代码中**。这样做存在密钥泄漏风险（例如代码泄露或被反编译）。推荐将 AK 配置在环境变量或独立的配置文件中，通过读取环境变量来使用。例如阿里云各语言 SDK 默认支持读取环境变量 `ALIBABA_CLOUD_ACCESS_KEY_ID` 和 `ALIBABA_CLOUD_ACCESS_KEY_SECRET` 来获取凭证。
- **最小权限原则**：为 RAM 用户授予尽可能小范围的权限，只赋予其调用所需资源的权限。例如仅调用查询接口的用户可授予只读权限，避免多余的写权限。
- **定期更换和禁用不必要的 AccessKey**：长期不更换的密钥存在更高泄露风险。可以制定密钥轮换策略，定期更新 AccessKey，并删除不再使用的密钥。主账号下 AccessKey 尽量维持0或1对，避免过多难以管理。
- **监控 AK 使用情况**：利用阿里云安全产品（如云安全中心）或 AccessKey 使用日志来监控异常使用。阿里云支持检测 AccessKey 泄露的情况并通知用户。一旦怀疑 AK 泄露，应立即禁用该密钥并排查影响。
- **使用临时安全令牌（STS）**：对于在客户端（如移动应用）直接调用阿里云 API 的场景，不应嵌入长期 AK，可考虑使用STS临时令牌来访问。STS凭证具有时效短、权限可控的优点，能降低泄露后风险。RAM角色可以颁发STS临时访问凭证供程序调用，具体可参考阿里云 STS 文档（属于进阶内容，这里不展开）。

总之，AccessKey 是 API 调用的敏感凭据，必须安全存储和传输，切勿在公共场合泄露。在服务器端调用阿里云API时也应做好密钥管理，例如将密钥放入安全的配置中心，而不是散布在代码仓库中。

各种调用方式（SDK、OpenAPI Explorer、HTTP 直调）：
阿里云为开发者提供了多种方式调用 OpenAPI，您可以根据需要选择：

- **使用阿里云各语言 SDK（推荐）**：官方提供完善的多语言 SDK，是集成阿里云 API 最简便可靠的方式。SDK 封装了请求的构造、签名、重试、错误处理等逻辑，开发者只需调用 SDK 提供的方法即可完成 OpenAPI 调用。SDK 调用返回结构化的响应对象，更易于解析和处理。目前阿里云提供的 SDK 覆盖 Java、Python、PHP、JavaScript/TypeScript、C#/.NET、Go 等主流语言。官方**强烈推荐**使用 SDK 调用 OpenAPI，以避免手动实现签名逻辑的繁琐和出错。
- **OpenAPI Explorer 在线调试**：OpenAPI Explorer 是阿里云提供的在线 API 调试工具。通过网页界面，您可以选择产品和接口，填写请求参数并发起实际调用，查看返回结果。使用 OpenAPI Explorer 无需手动配置 AccessKey，它会利用您登录账号的权限来调用 API（或使用临时授权）。OpenAPI Explorer 还会自动生成对应请求的代码示例（SDK 或 HTTP 请求），方便您参考集成。此外，其**错误诊断**功能可以根据返回的错误信息给出可能的原因分析和解决提示（这是 OpenAPI 开发者门户的自助诊断工具）。OpenAPI Explorer 适合在开发和测试阶段快速调试接口、查看请求和响应结构，以及学习如何使

用 SDK 调用某接口。需要注意实际生产环境中仍应以应用代码调用为主，Explorer 更多用于交互式的调试和学习。

- **HTTP 接口直调**：即不使用 SDK，直接构造 HTTP/HTTPS 请求来调用 OpenAPI。这种方式需要开发者**自行实现签名算法**并拼接请求，各语言均可使用标准HTTP库实现。直调适用于特殊场景（如不方便引入SDK的极简环境，或者需要在不支持SDK的语言中调用）。但它对调用者要求最高，需要仔细按照文档规范计算签名、设置正确的Header和参数。一旦签名字符串拼接有误，就会导致身份验证失败。因此，除非必要，一般不建议自行计算签名直调，而应使用SDK简化开发。如果确需手动直调，可参考上文“签名机制详解”部分确保签名正确，并使用工具（如 Postman、curl 或 OpenAPI Explorer 提供的示例）进行对照测试。

(说明：除此之外，阿里云还提供命令行工具CLI、资源编排ROS、Terraform等方式调用API。但这些方式本质上也是对OpenAPI的封装或批量调用，这里不展开。常规开发集成主要还是 SDK 或直接HTTP调用。)

多语言 SDK 使用说明（安装、配置、示例）：
阿里云官方SDK支持多种语言，并为每种语言提供详细的安装和使用指南。以下简要说明常见语言的 SDK 获取与调用：

- **Java SDK：** 阿里云 Java SDK 可以通过 Maven 等构建工具添加依赖获取。以调用 ECS API 为例，可以在项目的 pom.xml 的 `<dependencies>` 中加入如下依赖：

复制代码

```
<dependency>
    <groupId>com.aliyun</groupId>
    <artifactId>ecs20140526</artifactId>
    <version>5.3.0</version>
</dependency>
```

上述 artifactId “ecs20140526” 对应的是 ECS 产品2014-05-26版API的SDK模块，版本号5.x属于新版 2.0 SDK。添加依赖后，通过 Maven/Gradle 下载库。然后在代码中使用 SDK 提供的 Client 类进行调用。例如初始化客户端：

复制代码

```
import com.aliyun.ecs20140526.Client;
import com.aliyun.teaopenapi.models.Config;
// ...
Config config = new Config()
    .setAccessKeyId(System.getenv("ALIBABA_CLOUD_ACCESS_KEY_ID"))    //
从环境变量获取AK ID

.setAccessKeySecret(System.getenv("ALIBABA_CLOUD_ACCESS_KEY_SECRET"))
// 获取AK Secret
```

```
    .setEndpoint("ecs.cn-hangzhou.aliyuncs.com");  // 设置服务Endpoint，例如华东1接口地址
Client client = new Client(config);
```

如上，通过 Config 对象配置凭证和 Endpoint，并创建 ECS Client 实例。接着构造请求对象并调用接口，例如调用查询实例列表 `DescribeInstances`：

复制代码

```
DescribeInstancesRequest request = new DescribeInstancesRequest().setRegionId("cn-hangzhou");
DescribeInstancesResponse response = client.describeInstancesWithOptions(request, new RuntimeOptions());
System.out.println(response.body.getInstances());  // 打印返回的实例列表
```

SDK 会将 API 的返回映射为响应对象，如上述 `DescribeInstancesResponse`，其中包含请求的 RequestId 和具体数据。使用 Java SDK 时，还需注意异常处理，SDK 将错误分为 `TeaException`（业务错误）和 `TeaUnretryableException`（不可重试的底层错误）等，需要按情况捕获处理。

- **Python SDK：** 阿里云 Python SDK 可以通过 pip 安装对应产品的库。例如安装最新版本的 ECS SDK：

复制代码

```
pip install alibabacloud_ecs20140526
```

安装后，可在 Python 代码中使用 SDK 调用。首先初始化客户端，例如：

复制代码

```
import os
from alibabacloud_tea_openapi.models import Config
from alibabacloud_ecs20140526.client import Client as ECSClient

config = Config(
    access_key_id=os.environ.get('ALIBABA_CLOUD_ACCESS_KEY_ID'),
# 读取AK ID

access_key_secret=os.environ.get('ALIBABA_CLOUD_ACCESS_KEY_SECRET'),# 读取AK Secret
    endpoint='ecs.cn-hangzhou.aliyuncs.com'    # 设置ECS服务Endpoint
)
client = ECSClient(config)
```

然后构造请求并调用，例如调用 DescribeInstances 接口：

复制代码

```
from alibabacloud_ecs20140526 import models as ecs_models
request = ecs_models.DescribeInstancesRequest(region_id='cn-hangzhou')
response = client.describe_instances_with_options(request, None)
print(response.body.request_id, response.body.instances)
```

Python SDK 返回的 `response.body` 对象包含 RequestId 和结果数据。SDK 已处理了签名和请求发送，直接使用即可。对于可能出现的网络类错误，Python SDK 抛出 `UnretryableException`，业务错误抛出 `TeaException`，可分别捕获处理。

- **其他语言：**
  PHP: 可通过 Composer 安装，例如 `"alibabacloud/sdk"` 或特定产品包，然后按文档初始化 Client（配置 AccessKey 和 Endpoint），调用对应方法。
  JavaScript/Node.js: 使用 npm 安装阿里云 Node.js SDK，如 `npm install @alicloud/ecspan`（具体包名视产品而定），然后通过 `new ROAClient` 或 `RPCClient` 初始化（提供 accessKeyId、accessKeySecret、endpoint 等），再调用 `client.request('<Action>', params)` 或新版 SDK 提供的方法。
  .NET: 通过 NuGet 引入阿里云 SDK，例如安装 `AlibabaCloud.SDK.Ecs20140526` 包，然后使用其 Client 类，构造 Config 对象设置凭证和 Region Endpoint 后调用 API 方法。
  Go: 使用 `go get` 引入 SDK，例如 `go get github.com/alibabacloud-go/ecs-20140526/v2/client`，然后在代码中使用其 Client，配置 config 并调用API。

各语言 SDK 的用法风格略有不同，但总体流程一致：**安装 SDK -> 配置凭证和 Endpoint -> 构建请求参数 -> 调用 API 方法 -> 处理返回/异常**。详细的多语言示例代码可以参考阿里云官方文档或 OpenAPI 开发者门户提供的示例。利用 SDK，可以免去计算签名、拼接 HTTP 请求的繁琐工作，大幅降低接入难度和错误率。

请求/响应结构示例（JSON/XML）：
阿里云 API 的响应有统一的格式结构。无论请求成功或失败，返回中都会包含请求 ID（RequestId）用于唯一标识请求。下面给出一个典型 API 的请求和返回示例：

- *请求示例（HTTP GET）：*

复制代码

```
https://<产品域名>/?Action=DescribeInstances&Version=2014-05-26
&AccessKeyId=<您的AccessKeyId>
&Timestamp=2023-03-07T12:00:00Z
&SignatureMethod=HMAC-SHA1&SignatureVersion=1.0
&SignatureNonce=1234567890
&Format=JSON
&Signature=<计算出的签名值>
```

```
&RegionId=cn-hangzhou
```

该请求尝试查询华东1地域的实例列表（Action为DescribeInstances）。其中公共参数和业务参数已包括在查询串中。实际使用时 `<您的AccessKeyId>` 和 `<计算出的签名值>` 需替换为真实内容。

- *正常返回示例：* 如果调用成功（HTTP状态码2xx），返回内容根据请求中 Format 参数而定，JSON 或 XML 格式。假定仅展示通用部分，则：

XML 格式：

复制代码

```
<?xml version="1.0" encoding="UTF-8"?>
<DescribeInstancesResponse>
    <RequestId>4C467B38-3910-447D-87BC-AC049166F216</RequestId>
    <!-- 具体返回的数据内容，如实例列表等 -->
</DescribeInstancesResponse>
```

JSON 格式：

复制代码

```
{
    "RequestId": "4C467B38-3910-447D-87BC-AC049166F216",
    /* 具体返回的数据内容，如实例列表等 */
}
```

实际返回中，根节点名称通常是接口名+“Response”，其中包含 RequestId 和业务数据。如果数据量较大，返回JSON会以键值对呈现，XML则以相应结构的节点呈现。以上示例中结果数据部分省略，格式上 JSON 用花括号、XML 用标签，语义上都是一样的。官方文档示例多经过换行缩进美化，实际 API 返回不包含换行和空格。

- *错误返回示例：* 如果调用出错（HTTP状态码为4xx或5xx），则不会有正常结果数据，而是返回错误信息。返回体包含错误代码（Code）、错误消息（Message）、以及 RequestId，有时还包括 HostId。HostId表示请求访问的服务节点标识，用于阿里云内部追踪。下面是一个错误返回示例（假设请求缺失时间戳导致错误）：

XML 格式错误示例：

复制代码

```
<Error>
    <HttpStatus>400</HttpStatus>
    <Code>IllegalTimestamp</Code>
```

```
    <Message>The input parameter "Timestamp" that is required for
processing this request is not supplied.</Message>
    <RequestId>3B237773-BC2C-4BEA-95FC-319A1A5BAA68</RequestId>
</Error>
```

JSON 格式错误示例：

复制代码

```
{
    "HttpStatus": 400,
    "Code": "IllegalTimestamp",
    "Message": "The input parameter \"Timestamp\" that is required for
processing this request is not supplied.",
    "RequestId": "e85db688-a2d3-44ca-9790-4259f59e90d8"
}
```

可以看到，错误结构中 Code 表示错误类型，Message 提供了英文描述（有时还会有对应的中文提示），以及 RequestId 用于跟踪查询。当您无法确定错误原因时，可以通过工单联系阿里云并提供该 RequestId 和相关时间、HostId 等信息，以便支持人员快速定位问题。

常见错误码及排查建议：
在调用 OpenAPI 过程中，经常会遇到一些通用的错误响应。以下列出常见错误码含义并给出排查建议：

- **InvalidAccessKeyId.NotFound：** 指定的 AccessKey ID 不存在。通常是使用了错误的 AccessKey ID 或该 ID 已被删除。建议确认 AccessKey ID 填写是否正确，是否和您账号下有效的 AccessKey 匹配。如果是 RAM 用户，需确保使用的是该用户的 AK 而非主账号的（或反之）。另外，检查是否有多余空格或字符导致ID不匹配。
- **SignatureDoesNotMatch：** 签名不匹配错误。这意味着请求中的签名和服务端计算的签名不一致，身份校验未通过。常见原因包括使用了错误的 AccessKey Secret、签名算法步骤有误、参数编码排序不正确等。建议首先核对 AccessKey Secret 是否对应正确的 AccessKey ID 且无拼写错误。然后检查签名计算过程：确保所有参与签名的参数都正确排序编码，签名字符串拼接符合规范，特别注意 URI 编码细节、大小写、以及使用正确的哈希算法和密钥。必要时可使用官方提供的示例参数进行对比验证。如果仍有疑问，可参考阿里云官方关于签名不匹配的排查指南。
- **IncompleteSignature：** 请求的签名不符合阿里云标准。这通常发生在签名字符串不完整或构造错误的情况下。例如可能缺少必要的参数参加签名，或签名版本不对。解决方法与 SignatureDoesNotMatch 类似：严格按照签名算法步骤重新构造 StringToSign 并计算签名，确保没有遗漏参数或顺序错误。该错误有时和 SignatureDoesNotMatch含义接近，均应从签名算法实现上找原因。

- **MissingParameter：** 缺少必需的参数。“The parameter <ParameterName> is needed but not provided” 表明您的请求缺少某个接口要求的字段。出现此错误时，根据错误消息指示，检查 API 文档中该接口的必填参数列表，补充遗漏的参数并重试。常见的是 Version、Action、Timestamp、RegionId 等不可缺少的参数未提供或者拼写错误。
- **InvalidParameter：** 参数格式或值不合法。“The specified parameter <Name> is not valid” 提示某参数值超出允许范围或格式不正确。需核对文档中该参数的取值要求。例如 RegionId 拼写错误或不存在、枚举型参数提供了未支持的值等都会引发此错误。纠正参数值后再调用。
- **IllegalTimestamp：** 时间戳不合法。通常是 Timestamp 参数缺失或格式错误，或者时间超出了允许范围导致。比如请求时间与服务器时间相差超过有效窗口（一般±15分钟）就会返回 “请求过期” 相关错误。解决方法：确保请求携带 Timestamp 参数，格式严格符合 ISO8601 UTC，并且系统时钟准确。使用 NTP 保持服务器时间同步可以避免该问题。
- **InvalidAccessKeyId.Inactive：** AccessKey ID 未启用或已被禁用。这种情况下需要更换有效的 AK，或到 RAM 控制台检查该 AccessKey 状态，将其启用。
- **Forbidden.NoPermission：** 没有权限执行该操作。也表现为错误码 “NoPermission” 或者 HTTP 403 Forbidden 且消息包含 “You are not authorized to operate on the specified resource” 等。此错误说明当前使用的 AccessKey 没有调用目标 API 的权限。可能的原因：如果是 RAM 用户，未被授予对应云产品API的权限；如果是主账号，某些资源属于别的账号或特殊权限限制。排查方法：到 RAM 控制台确认该用户是否已附加正确的授权策略。例如调用 ECS API 需要授予用户 `AliyunECSFullAccess` 或更细粒度的策略。如未授权则授权后重试。对于角色扮演的临时凭证，要确保角色有相应权限且 Token 未过期。
- **Throttling.User / RequestLimitExceeded：** 请求被限流。阿里云对部分 API 有调用频率限制，如果短时间内请求过多会返回此错误，一般HTTP 429或400状态，提示超过频率限制。解决方案：实现重试退避(backoff)，降低调用频率，或联系阿里云申请提高配额（如服务支持自定义QPS上限的，可以提工单咨询）。
- **ServiceUnavailable/InternalError 等5xx错误：** 服务器内部异常。这通常是阿里云侧的问题，如服务暂时不可用或内部处理错误。开发者能做的是稍等后重试请求。如果持续出现，可联系阿里云支持并提供 RequestId 以协助调查。

此外，在本地调用 API 时如果**迟迟没有响应或连接失败**，需考虑网络层面的原因。例如防火墙阻断、DNS 无法解析 Endpoint、或网络不通等。确保服务器可以连通阿里云接口域名，可通过 ping 或 curl 命令测试连通性。如果网络正常但仍超时，可以适当延长 SDK 或 HTTP 客户端的超时时间。设置合理的重试机制也有助于提升调用的可靠性。总之，大部分错误都可以根据错误码定位到问题根因，针对性解决。

接入流程完整示意（注册、开通、调用、调试、运维）：
综合以上内容，这里将典型的 API 接入流程整理如下，方便从零开始理解如何使用阿里云 OpenAPI：

1. **注册阿里云账号：** 首先需要一个阿里云账户用于发起 API 调用。如果尚未注册，请前往阿里云官网注册并完成实名认证等（某些云产品要求账号已实名认证才能调用其API，如短信服务等）。主账号一般用于管理和授权，不直接用于程序调用。

2. **开通目标云产品服务：** 某些云产品需要先在控制台开通服务才能使用 API。例如 NLP、自定义AI服务等需在控制台订阅或购买后才开放调用权限。确保您已在阿里云控制台启用了要调用的云产品（有些基础服务如ECS默认开通，但某些增值服务需要手动开通）。开通后，参考该产品的价格和配额说明，准备好相应资源（例如购买实例，或确保有可用免费额度等）。
3. **获取 AccessKey 凭证：** 如前文所述，在调用 API 前需要取得访问凭证 AccessKey ID/Secret。进入阿里云控制台的RAM用户管理页面：
   - 若用主账号直接调用（不推荐），可在 “安全设置” 中创建主账号 AccessKey。但强烈建议**改用 RAM 用户**。先创建一个RAM子用户，授予所需权限，然后在该用户详情中创建 AccessKey。
   - 获得 AccessKey 后，务必保存好 AccessKey ID 和 Secret。**切记不要**将 Secret泄露或硬编码在公开场合。您可以在服务器环境变量中设置 `ALIBABA_CLOUD_ACCESS_KEY_ID` 和 `ALIBABA_CLOUD_ACCESS_KEY_SECRET`，在程序中读取它们，这样不用把密钥写死在代码里。
4. **（可选）为 RAM 子账号授权：** 如果使用 RAM 用户的 AccessKey 调用 API，确保该用户拥有调用目标 API 所需的权限。例如调用某个云服务的API，需要为用户授予对应产品的API访问权限（系统策略或自定义策略）。未授权的RAM用户调用将返回权限不足错误（如 NoPermission）。授权可通过RAM控制台将预定义策略附加给用户，或创建自定义策略限定其资源和操作范围。遵循最小权限原则配置，以保证安全。
5. **了解并选择 API 接口：** 登录阿里云官网文档中心或 OpenAPI 开发者门户，查找目标产品的 API 文档。阅读该产品的接口说明和调用指南，确认您需要调用的具体 Action 或 REST接口路径。例如，通过产品文档的 API 列表找到所需的功能对应的 API 名称和参数要求。许多产品在控制台或文档中也直接提供调用示例和 SDK 代码示例，充分利用这些资料来正确构造请求。
6. **正式发起调用：** 编写代码使用选定方式调用 API。此时有几种实现途径：
   - **使用 SDK 调用：** 按照之前介绍，初始化 SDK Client（配置好 AccessKey 和 Endpoint 等），然后调用 SDK 提供的方法。SDK 会负责发送请求并拿回结果。确保传入的参数正确无误。对于网络不稳定的情况，可以配置SDK的重试和超时参数来提高健壮性。
   - **使用 OpenAPI Explorer 试调：** 在真正写代码前，可以用 Explorer 来测试一下所选 API 是否能成功返回结果。这有助于您验证参数是否正确、账号权限是否满足。如果 Explorer 调用返回正常，说明您的账号和接口都准备就绪，可以参考其生成的代码示例来撰写程序代码。
   - **HTTP 工具调用：** 您也可以用 curl/Postman等工具构造HTTP请求测试签名是否正确。在生成代码前，这种方式主要用于调试签名过程。当您完全按照文档计算签名并调通后，再将相同逻辑写入代码。
7. **调试与错误排查：** 在开发过程中，如果调用失败，要根据返回的错误信息进行调试。利用上文提供的常见错误码指南，对照错误 Code 和 Message 确定问题。例如出现参数错误，就检查请求参数拼装；出现签名错误，就重点调试签名计算过程；出现权限错误，则检查 RAM 授权等。您也可以使用阿里云提供的**API自助诊断工具**（OpenAPI Portal 中的

诊断功能）输入错误信息，获取可能的原因分析。调试时建议输出日志，包括请求ID、参数等，以便重现问题或提供给官方支持查看。

8. **运维监控：** 当将 API 集成到生产系统后，需要做好后续的运维：
   - **日志和审计：** 建议打开阿里云的操作审计（ActionTrail）服务，跟踪记录该账号下的 OpenAPI 调用日志。这有助于在出现安全或异常情况时追溯调用记录。
   - **配额和性能：** 关注 API 调用配额，如每日次数、每秒请求数QPS限制等。如果达到上限会收到限制错误，需要评估是否申请提升配额或在应用侧做好限流。
   - **错误告警：** 可以在代码中对调用结果进行检查，若返回错误码，及时通过日志或监控告警通知运维人员。例如连续出现某错误码可能表示系统配置或阿里云服务有问题，需要介入调查。
   - **密钥管理：** 定期轮换 AccessKey，更新部署中的凭证；如发现可疑调用及时更换密钥。对于不再使用的 RAM 用户或AccessKey要及时删除。
   - **SDK升级：** 关注阿里云 SDK 和 API 的更新公告。SDK 版本更新可能包含漏洞修复或新功能，定期升级有助于系统稳定。云产品API版本变更时也要同步调整使用的 Version 参数或SDK版本。

淘宝开放平台与服务市场合规要点

在对接阿里AI能力的同时，如果应用涉及淘宝电商平台（例如作为淘宝开放平台的开发者，或在淘宝商家服务市场提供应用服务），必须遵守淘宝平台及开放生态的一系列规则规范，确保业务合规运营。以下梳理淘宝开放平台和商家服务市场的主要规则要点：

- **淘宝平台通用规则**：所有淘宝开放生态的参与者首先必须遵守《淘宝平台规则》及相关协议的要求。这些规则旨在维护一个“开放、透明、分享、责任”的新商业文明环境，保障各方合法权益，维护平台正常秩序。商家服务市场的服务商和商家用户都被要求**遵守淘宝平台已有规则以及本规范的规定**，如平台已有规定则依其执行，平台未规定或本规范有特殊要求的，则按本规范执行。同时，平台倡导诚信经营，要求服务市场用户诚实守信、遵纪守法，提升服务品质和信誉，共同营造公开、公正、文明、有序的生态环境。
- **服务市场管理规范**：这是对淘宝**商家服务市场**（即为商家提供软件工具、运营服务等增值服务的平台）的整体运营要求。规范明确服务商入驻、经营、退出的规则。例如，服务市场用户需签署《阿里巴巴商家服务市场服务协议》并按照规范完成入驻，交纳保证金等。服务商在服务市场的行为若发生违规，平台可采取**包括下架服务、冻结新签、停止分成、服务清退**等措施处理。比如，“服务下架”指应用将无法在服务市场被新用户订购；“新签冻结”则暂停新用户订购但不影响已订购用户使用；情节严重的还可能“服务冻结或清除”，终止应用在市场上运营。规范还规定了服务商主动退出的条件（如须先下架所有在售服务等）。总体来说，服务市场管理规范覆盖了从服务商准入资质、服务发布要求、运营行为守则、违规处理、退出机制等全生命周期的管理要求，确保服务市场健康运转。
- **安全违规处理规范**：该规范聚焦开放平台及服务市场中的**安全类违规**，对开发者和应用的违规行为进行了分类和处罚标准说明。开发者应遵守《淘宝合作伙伴开发者协议》《开放平台基础规范》《淘宝规则》等，否则平台有权视违规情节轻重对应用或开发者采取一项或多项处理措施。违规行为分级包括例如：

- *恶意违规且情节特别严重*：如开发者发现并恶意利用淘宝平台漏洞，或其应用造成超过500笔用户投诉、超过200万人民币的损失等重大影响。此类最高可被记12分严重违规，可能导致应用强制清退、保证金扣罚等。
- *严重违规*：如应用造成较大范围用户受影响，产生较多投诉或损失（低于上述标准），可被记6分等处罚。
- *应用安全违规*：如开发者对外泄露其应用的高危端口、管理接口、源代码目录等敏感信息，造成安全隐患。
- *内容安全违规*：如应用提供的功能或通过API发布的内容中含有违法违规的信息（涉黄、涉政等），违反国家法律法规。

对不同违规类型，处理标准包括**违规记分**、**保证金处罚**和其他限制措施。例如，针对严重违规的开发者，平台可冻结并没收其已缴纳的违规保证金（不同服务类型的应用有5万/20万/50万人民币不等的保证金梯队标准）；同时根据累计违规扣分情况，可能限制开发者API调用、下架应用直至清退开发者资格等。平台还会定期在规则中心公示违规名单和处理结果，以起到警示作用。因此，作为开放平台开发者，一定要在安全层面合规：**不得恶意利用平台系统、不得泄露敏感数据、不得传播违法内容**，保障用户和平台利益安全。

- **争议处理规则**：此规则用于规范商家服务市场中**交易纠纷的处理流程**。当商家（购买服务的一方）与服务商（提供服务的一方）因服务订购或履约产生争议时，双方应本着公平诚信原则优先协商解决。如果协商无法达成一致，任一方可申请平台介入调解。平台作为独立第三方，有权要求争议双方提供证明证据，并单方判断证据效力。双方必须按照平台要求在规定时限内提交凭证，**若无正当理由拒不提供证据，平台将依据现有证据做出裁决**。平台介入后做出的调解结论为最终结果，各方应遵照执行。

平台处理争议的规则还包括退款和赔付的判定标准。例如：

- **服务合法性**：若服务商交付的服务本身违反法律法规，或违反平台管理规范中关于禁止提供的服务/应用的规定，经查实交易应做退款或部分退款处理。
- **描述与履约**：服务商必须如实描述其服务或应用，包括功能、缺陷、技术局限等。如果存在与实际不符的夸大宣传，或对同一服务有前后不一致的描述，平台会按照*有利于客户*的原则解释争议并支持客户的退款请求（除非双方另行协商达成其他一致）。简而言之，**服务商的宣传承诺和实际交付必须一致**，否则需承担相应责任。
- **履约中止**：如果服务商在服务期内单方面中止或严重延期提供服务且未与客户协商同意，平台通常支持客户申请退款并对服务商进行处理（具体视服务类型而定，如代运营服务不得承诺交易额指标，培训课程不得无故取消延期等，这些在各细分类目规则中亦有规定）。

总之，争议处理规则保障了商家在购买增值服务过程中的权益：服务商**必须诚信履约，按约定内容和标准提供服务**，出现纠纷时平台居中裁决亦偏重保护消费者权益。这提醒服务商在售前明确服务边界、不夸大承诺，售中妥善履约，售后积极配合解决问题，以免因纠纷被退款处理甚至影响信用。

- **个人信息保护规范**：随着《中华人民共和国个人信息保护法》等法规施行，淘宝开放平台也制定了合作伙伴需遵守的个人信息保护规范。合作伙伴（包括开放平台开发者和服务市场服务商）应充分认识到**个人信息保护是企业长期发展的基石**，承诺遵循以下原则保障用户个人信息权益：

> a. **强化意识，落实责任**：坚持客户第一，重视数据安全与用户隐私保护，建立牢固的个人信息保护意识，切实履行企业主体责任，保障消费者合法权益。
> b. **用户为本，合法合规**：强化用户权益保护，建立完整的个人信息保护风控体系，确保**仅在合法、正当、最小必要的范围内收集和使用个人信息**。未经用户明示授权，不得处理超出服务必要范围的个人数据，尤其**敏感信息必须取得用户单独同意并实施严格保护措施**。
> c. **技术保障，安全防护**：持续投入先进安全技术，提高隐私保护能力。例如采用差分隐私、多方安全计算等手段保护数据，防止个人信息泄露和滥用。
> d. **制度建设，持续改进**：完善企业内部的数据治理和隐私保护制度，配备专业人员、定期培训演练，不断提升合规管理水平。

合作伙伴在为用户提供应用服务过程中涉及的个人信息处理活动，**必须符合中国法律法规要求**（如《个人信息保护法》《数据安全法》等）。同时，应参照国家标准《信息安全技术 个人信息安全规范》（GB/T 35273-2020）等实行高标准的保护措施。平台运营方（如淘宝网）自身已制定并执行了严格的隐私政策和数据安全措施，合作伙伴**必须制定并执行不低于平台标准的隐私政策和安全措施**，确保与平台一同满足监管要求。

此外，规范要求合作伙伴建立内部**个人信息保护责任制**，指定专门负责人和部门持续开展个人信息和信息安全管理。合作伙伴应**至少每年开展一次自查和安全评估**，检查信息系统安全和各项措施落实情况，发现不足及时整改。对于个人信息处理活动，建议定期进行**个人信息保护影响评估（PIA）**并留存记录不少于3年。总之，在淘宝开放生态中处理任何用户数据，都必须依法取得必要授权，确保数据最小化使用和安全储存，不得将用户个人信息用于非法目的或违反公序良俗的用途。

- **应用质量管理规范**：淘宝开放平台建立了生态应用质量管理机制，以保障入驻应用的服务质量和用户体验。服务市场的应用不仅要功能合规、安全可靠，还需在**性能、稳定性和用户满意度**方面达到要求。平台会对应用的质量指标进行监控，包括应用的可用性、响应速度、出错率等技术指标，以及**用户评价和违规记录**等运营指标。如果应用长期不更新且存在严重BUG、频繁宕机，或用户评分过低、投诉较多，平台可能介入处理，例如下架问题应用、要求限期整改。针对严重违反质量规范或用户体验极差的应用，平台有权清退其服务市场资格。

为了促使开发者重视质量，开放平台还实施了**违规公示制度**，定期公布服务商与开发者的违规情况。例如，每月都会发布当期**服务商/开发者违规公示**名单，曝光违规行为和处理结果。这种透明机制督促所有开发者引以为戒，不断优化自己的应用。开发者在日常运营中应当及时响应用户反馈、修复故障问题，避免因为服务不稳定或不符合描述而触犯质量管理规范。只有持续提供高质量、高可靠性的应用服务，才能在淘宝生态中获得更高的信用和更多业务机会。

- **商品发布规则要点**：如果业务涉及在淘宝网上架商品（例如利用AI技术批量发布商品信息或审核内容），还需了解**淘宝商品管理与发布规范**。淘宝于2020年整合发布了《淘宝网商品发布规范》，对商品上架的信息要求、类目管理、特殊商品限制等做出了明确规定。以下为关键点摘要：
  - **账户资质**：卖家必须通过平台实名认证并绑定支付宝账户，确保账号合法有效，才能上架商品进行交易。这是电商经营的基本门槛，资料不实或未完成认证的，将无法正常发布商品。
  - **商品信息真实完整**：商品的标题、描述、规格参数、价格等信息都必须填写完整、准确，无虚假夸大或误导性描述。标题长度不得超过30个汉字（即60字符），严禁包含违禁词语或不当宣传。建议标题格式包含品牌、品类和核心卖点，以清晰传递商品信息。商品详情描述中亦不得欺骗或隐瞒，与实际商品相符是硬性要求。
  - **图片规范**：商品图片要求清晰、美观，能够真实反映商品外观。主图建议采用1:1正方形构图，推荐尺寸1440×1440像素，并提供多角度高清展示（最多可上传5张主图）。禁止使用模糊失真、以次充好的图片，更不允许盗用他人图片或在图片中嵌入联系方式等导流信息。图片不符要求的商品将被限制展示或下架处理。
  - **正确选择类目**：商品必须发布在**正确的类目**之下，不得错放类目。淘宝对商品类目的管理非常严格，若将商品发布在不匹配的类目，平台会强制下架并按情节轻重给予处罚，影响店铺整体信用。卖家应根据商品属性选择最贴切的类目（例如童鞋应发布在童鞋/婴儿鞋类目下，而非成人鞋类目），以确保商品曝光和管理的正确性。
  - **禁限售商品**：淘宝平台**严禁发布违法违规或侵害他人权益的商品**。包括但不限于各类国家法律明令禁止流通的商品（如非法药品、武器等）、侵权假冒商品、违反社会公序良俗的物品等。对于此类**违禁品或侵权商品**，一经发现平台将依据规则予以下架、扣分等严厉处罚；情节特别严重的，可能直接关闭店铺。卖家务必熟知禁售品目录，避免触碰红线。
  - **发布数量限制**：为了防止滥发信息，淘宝会根据店铺信用等级和经营类目，对可发布商品的数量设定上限。信用等级越高，允许发布的商品数量越多。新卖家应遵循这一限制，避免因超量发布被系统拦截。若需要提高上架数量，可以通过提升店铺信用或向平台申请等途径解决。
  - **违规发布处理**：商品发布规范还列出了诸多违规情形及处理办法，例如**重复铺货**（同一商品恶意重复发布）、**标题滥用关键词**、**信息不符**（描述与实际不符）等都会被视为违规，轻则降权或删除链接，重则扣分处罚甚至店铺清退。这些细则在淘宝规则中心都有详细说明，卖家和开发者都应仔细阅读以避免违规。